

Please do Early Module
Feedback today!

Q: questions or comments?

Lecture 10: HTTP API Examples (with OpenAPI and Java)

Distributed Systems

Chris Greenhalgh

Contents

- HTTP APIs recap
- Swagger / OpenAPI
- Mailbox API specification (example)
 - Form/query-style
 - REST-style
- Spotify API specification (example)
 - Authentication
 - Examples
- Java HTTP clients
- Java HTTP servers

Note, this lecture is mostly illustrative examples.
The main thing to look at is how HTTP APIs can be defined, and what this might mean for their implementation, e.g. how paths, URLs and body types can be used for different operations and styles.

HTTP APIs Recap

- HTTP Application Programming Interfaces (**APIs**) can be defined in various ways
 - Generally specifies combinations of URL, HTTP method, authentication, body, response and error signalling
- HTTP can be used in different ways (idioms) in a distributed system, e.g. [more in this lecture...]
 - **Form- & query**-style - Designed to be easily supported by server-side scripting frameworks developed for regular browser client (e.g. CGI, PHP)
 - **RPC** (Remote Procedure Call)-style - Starts with RPC concepts (e.g. interfaces, methods), NOT HTTP [we don't do any more on this style – dated/specialist]
 - **REST** (REpresentational State Transfer) - Starts with HTTP concepts, i.e. resources (URLs) & HTTP methods

REST vs form/query-style APIs Summary

(illustrations follow...)

REST-style

- URL identifies a specific **resource**
 - e.g. a database table or a single database record
 - Path may include: resource type; resource identifier(s); view name, facet or relationship
- HTTP method identifies the **operation** to perform
 - GET = read, PUT = write, POST = create/append, DELETE = delete, PATCH = edit

Form- and Query-style

- URL identifies the server and **operation** to perform
 - E.g. script or procedure to execute
- Query parameters or request body provide arguments, e.g. IDs
- HTTP method is
 - GET for normal navigation or data requests without side-effects, or
 - POST, for operations with side-effects

OpenAPI / Swagger

OpenAPI / Swagger overview

- “The OpenAPI Specification provides a formal standard for describing HTTP APIs.” - <https://www.openapis.org/>
- “The OpenAPI Specification (OAS) is a vendor-neutral description format for HTTP-based remote APIs. It was originally based on the Swagger 2.0 Specification, donated by SmartBear Software in 2015.”
- Widely adopted
- Good tool support, e.g. code-generation
- Aims to describe common use cases efficiently

OpenAPI Description (OAD)

- An OpenAPI Description (OAD) is defined in a YAML or JSON file, inc.
 - “openapi” – OpenAPI specification version
 - “info” – interface metadata including title & version (of interface)
 - “paths” – endpoints map, e.g. “/dosomething” ->
 - HTTP method map, e.g. “get” ->
 - Operation object, including
 - metadata (summary, description, tags),
 - Security (authentication) details
 - Request body definition
 - Responses map, e.g. “200” ->
 - Response body definition
 - “components” (optional) – common definitions, including e.g.
 - “schemas” – map of common JSON schema definitions
 - “servers” (optional) – list of actual server base URLs

You don't need to memorise the details, but you should understand the basic idea

OpenAPI tool support

- There are many tools to support OpenAPI/Swagger, e.g.
 - <https://swagger.io/tools/open-source/>
 - Custom editors,
 - e.g. <https://editor.swagger.io/>
 - Code generators, for clients and servers in various languages and frameworks
 - API UIs
 - E.g. see editor (above)- righthand panel (import an example API first), or
 - <https://developer.spotify.com/documentation/web-api/reference/get-an-artist> (customised & extended)

Mailbox API Example

Not directly examinable: Illustrative examples of the principles!

Recap & reference:

Java Remote interfaces *Mailbox*

```
import java.rmi.*;

public interface Mailbox extends Remote {
    public Vector<Header> list()
        throws RemoteException;
    public Message getMessage(int msgid)
        throws RemoteException;
    public void post(Message message)
        throws RemoteException;
}
```

```
public class Header
implements Serializable {
    public int id;
    public String subject;
}

public class Message
implements Serializable {
    public Header header;
    public String body;
}
```

Post message, form-style

- Path is operation name (“post_message”)
- Method is POST
 - => run command with side-effects
- Request body is form encoding of message fields
 - (subject, body – id is server-assigned)
- Errors are (still) signalled with HTTP status codes

```
/post_message:
  post:
    tags:
      - mailbox
    summary: Post a new message
    description: Post a new message
    requestBody:
      description: Create a new message in the mailbox
      content:
        application/x-www-form-urlencoded:
          schema:
            $ref: '#/components/schemas/MessageForm'
        multipart/form-data:
          schema:
            $ref: '#/components/schemas/MessageForm'
    responses:
      '200':
        description: Successful operation
      '400':
        description: Invalid Message supplied
```

Post message, REST-style

- Path is resource type (“messages”)
- Method is POST
 - => create a new resource
- Request body is JSON encoding of message
- Errors are signalled with HTTP status codes

```
paths:
  /messages:
    post:
      tags:
        - mailbox
      summary: Post a new message
      description: Post a new message
      requestBody:
        description: Create a new message in the mailbox
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Message'
      responses:
        '200':
          description: Successful operation
        '400':
          description: Invalid Message supplied
```

Types

schemas:

Header:

type: object

properties:

id:

type: integer

format: int64

examples: [1]

subject:

type: string

examples: ['something or other']

Message:

type: object

properties:

header:

\$ref: '#/components/schemas/Header'

body:

type: string

examples: ['This happened, and then that']

MessageForm:

type: object

properties:

subject:

type: string

examples: ['something or other']

body:

type: string

examples: ['This happened, and then t

List message headers, form-style

- Path is operation name
("get_message_headers")
- Method is GET
 - => run command WITHOUT side-effects
- Response body is JSON encoded of array of headers
- (nothing much to go wrong)

```
/get_message_headers:  
  get:  
    tags:  
      - mailbox  
    summary: Get all message headers  
    description: Get all message headers  
    responses:  
      '200':  
        description: Successful operation  
        content:  
          application/json:  
            schema:  
              type: array  
              items:  
                $ref: '#/components/schemas/Header'
```

List message headers, REST-style

- Path is resource type (“headers”)
 - NB the list method returns headers only
 - An alternative might be type and filter, e.g. "/messages/headers" but that could be ambiguous
- Method is GET
 - => read
- Response body is JSON encoded of array of headers
- (nothing much to go wrong)

```
/headers:  
  get:  
    tags:  
      - mailbox  
    summary: Get all message headers  
    description: Get all message headers  
    responses:  
      '200':  
        description: Successful operation  
        content:  
          application/json:  
            schema:  
              type: array  
              items:  
                $ref: '#/components/schemas/Header'
```

Get message, form-style

- Path is operation name (“get_message”)
- Query parameter (“id”) is message ID
- Method is GET
 - => run command WITHOUT side-effects
- Body is JSON encoding of message
- Errors are (still) signalled with HTTP status codes
 - E.g. 400, 404

```
/get_message:  
  get:  
    tags:  
      - mailbox  
    summary: Get details of specified message  
    description: Get details of specified message  
    parameters:  
      - name: id  
        in: query  
        description: ID of Message to return  
        required: true  
        schema:  
          type: integer  
          format: int64  
    responses:  
      '200':  
        description: successful operation  
        content:  
          application/json:  
            schema:  
              $ref: '#/components/schemas/Message'  
      '400':  
        description: Invalid ID supplied  
      '404':  
        description: Message not found
```


Get message, REST-style

- Path is resource type (“messages”), and message ID
- Method is GET
 - => read
- Response body is JSON encoding of requested message (if found)
- Errors are signalled with HTTP status codes
 - E.g. 400, 404

```
/messages/{messageId}:  
get:  
  tags:  
    - mailbox  
  summary: Get details of one message  
  description: Get details of one message  
  parameters:  
    - name: messageId  
      in: path  
      description: ID of Message to return  
      required: true  
      schema:  
        type: integer  
        format: int64  
  responses:  
    '200':  
      description: successful operation  
      content:  
        application/json:  
          schema:  
            $ref: '#/components/schemas/Message'  
    '400':  
      description: Invalid ID supplied  
    '404':  
      description: Message not found
```

Mailbox API Summary

Form-style

- Path is operation name
 - Query parameter is argument if required (e.g. ID)
- Method is GET (read-only operation) or POST (read-write)
- Request body is (typically) form-encoded
- Response body is JSON/etc encoded output of operation

REST-style

- Path is resource type and/or ID
 - (Query is not used)
- Method is GET (read), POST (create/add)
 - Or PUT (write) or DELETE (delete)
- Request body is JSON/etc encoding of resource value
- Response body is JSON/etc encoding of resource value

Spotify API Specification

Not directly examinable: Illustrative examples of the principles!

Spotify Authentication

- The Spotify API requires a valid **OAuth2 access token** for every request
 - Provided as value of “Authorization” header
 - E.g. “Authorization: Bearer 1POdFZRZbvb...qqillRxMr2z”
 - Otherwise it will fail:
 - 401 Unauthorized – no token, or the token has expired
 - 403 Forbidden – the authenticated user is not allowed to do this
- So the OAuth2 protocol must be used by the client first to
 - Allow the user to **authenticate** directly with (in this case) Spotify
 - And **grant** the client permission to access Spotify on their behalf
 - Which is represented by an **access token** returned to the client to use
 - (or a **refresh token**, which can be used to get an access token)
 - (A **token** is a digitally signed representation of who you are and/or what you can do; anyone with a copy can use it until it expires, so it has to be kept secret!)

For example:

<https://developer.spotify.com/documentation/web-api/reference/get-an-artist?id=>

The screenshot shows the Spotify for Developers documentation page for the 'Get Artist' endpoint. The page is divided into three main sections: a left sidebar, a central content area, and a right sidebar.

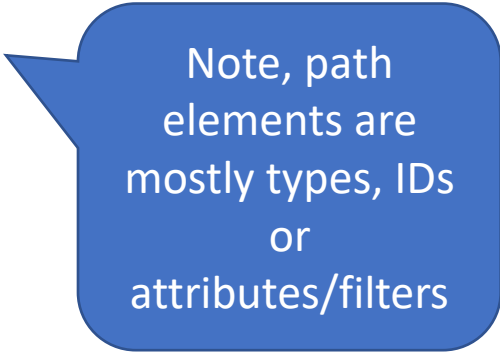
Left Sidebar: Contains a navigation menu with 'Web API' at the top, followed by 'Overview', 'Getting started', 'Concepts', 'Tutorials', and 'How-Tos'. Below these is the 'REFERENCE' section, which includes 'Albums', 'Artists' (selected), 'Audiobooks', 'Categories', and 'Chapters'. Under 'Artists', 'Get Artist' is highlighted.

Central Content Area: The title 'Get Artist' is displayed with an 'OAuth 2.0' badge. Below the title is a description: 'Get Spotify catalog information for a single artist identified by their unique Spotify ID.' A section titled 'Important policy notes' lists three points: 'Spotify content may not be downloaded', 'Keep visual content in its original form', and 'Ensure content attribution'. The 'Request' section shows a 'GET' method and the endpoint '/artists/{id}'. A parameter 'id' is listed as a 'string' and 'Required', with an example value: '0TnOYISbd1XYRBk9myaseg'.

Right Sidebar: The 'ENDPOINT' section shows the URL 'https://api.spotify.com/v1/artists/{id}'. Below this is a 'Try it' button. The 'REQUEST SAMPLE' section provides a cURL command: 'curl --request GET \ --url https://api.spotify.com/v1/artists/0TnOYISbd1XYRBk9myaseg \ --header 'Authorization: Bearer 1PodFZRZbvb...qqillRxMr2z''. The 'RESPONSE SAMPLE' section shows a JSON response snippet: '{ \"external_urls\": { \"spotify\": \"string\" },'.

Some Spotify operations (generally RESTful)

- GET /artists/{id} – get specified artist
- GET /artists/{id}/albums – get their albums
- POST /users/{user_id}/playlists – create a new playlist for specific user
- GET /users/{user_id}/playlists – get a user's playlists
- GET /playlists/{playlist_id} – get a specific playlist
- PUT /playlists/{playlist_id} – change a specific playlist
- POST /playlists/{playlist_id}/tracks – add tracks to playlist
- DELETE /playlists/{playlist_id}/tracks – delete tracks from playlist
- GET /search?type=...&q=... – search for matching entities



Note, path elements are mostly types, IDs or attributes/filters

Java HTTP Clients

Not directly examinable: Illustrative examples of the principles!

Note, Java tends to be pretty verbose and tends to feel a bit unwieldy for these kinds of things (unless using RPC-style wrappers).

JavaScript doesn't have threads so is mostly event-driven instead.

HTTP Client API

- Client needs to:
 - Specify URL, method, headers and request body
 - Marshall data to request body as required
 - Initiate HTTP request (wait for response)
 - Handle network failure, or if successful...
 - Check and handle HTTP status/error, and depending on status...
 - Unmarshall response body as required and
 - Handle response

E.g. Simple GET

Using Apache Fluent v4.5

`org.apache.http.client.fluent.Request;`
<https://hc.apache.org/httpcomponents-client-4.5.x/current/tutorial/html/fluent.html>

2. Method

1. URL

6. Success
result

3. Execute
(blocking);
throw exception
on network
failure

4. Check HTTP
status

5. Read (&
unmarshall?)
response body

```
// perform HTTP request
String result = Request.Get("http://www.cs.nott.ac.uk/~pszcmg/G52DSY/static.html")
    .execute().handleResponse(new ResponseHandler<String>() {
        // handle response
        public String handleResponse(final HttpResponse response) throws IOException {
            // check HTTP status
            StatusLine statusLine = response.getStatusLine();
            System.out.println("Status: "+statusLine);
            if (statusLine.getStatusCode() != 200) {
                throw new HttpResponseException(statusLine.getStatusCode(),
                    statusLine.getReasonPhrase());
            }
            // parse response body
            HttpEntity entity = response.getEntity();
            if (entity == null) {
                throw new ClientProtocolException("Response contains no content");
            }
            String body = EntityUtils.toString(entity);
            // return result
            return body;
        }
    });
```

E.g. GET with query parameters and headers

```
String name = args[0];
// perform HTTP request
URI uri = new URIBuilder("https://api.spotify.com/v1/search")
    .setParameter("type", "artist")
    .setParameter("q", name)
    .setParameter("limit", "2").build();
JSONObject result = Request.Get(uri)
    .addHeader("accept", "application/json")
    .addHeader("authorization", "Bearer "+args[1])
    .execute().handleResponse(new ResponseHandler<JSONObject>() {
        // handle response
    })
    ...
```

The diagram consists of three grey callout boxes with pointers. The first box, labeled 'Base URL', points to the string 'https://api.spotify.com/v1/search' in the URIBuilder constructor. The second box, labeled 'Query Parameters', points to the 'setParameter' calls for 'type', 'q', and 'limit'. The third box, labeled 'Headers', points to the 'addHeader' calls for 'accept' and 'authorization'.

API-specific clients

- You could encapsulate this in an API-specific client library, or
- OpenAPI code generators can create more RPC-like client libraries automatically from the OAD, e.g. (from the Mailbox API example)

```
public class MessageApi {  
    ...  
    /**  
     * Post a new message  
     * Post a new message  
     * @param body Create a new message in the mailbox (optional)  
     * @throws ApiException If fail to call the API, e.g. server error or cannot deseriali  
     */  
    public void messagePost(Message body) throws ApiException {  
        messagePostWithHttpInfo(body);  
    }  
}
```

Java HTTP Servers

Not directly examinable: Illustrative examples of the principles!

Personally I'd use JavaScript (Node.js), python or Go rather than Java nowadays for a web server, but some enterprise systems still do :)

Java HTTP server implementation

- You can build a HTTP Server in Java using...
- Raw TCP Sockets
 - You just have to implement all the IETF/W3C standards :-)
- `package com.sun.net.httpserver`
 - A "JDK internal" web server class
- Jakarta EE (formerly Java EE) Servlet API
 - Java standard framework for HTTP request handlers (and similar)
 - With or without various additional supporting frameworks, e.g. Spring
- Other proprietary or community Java frameworks,
 - e.g. Spring WebFlux reactive (i.e. non-blocking event-driven) web stack

Java Server Configuration requirements

- The web server code/configuration will need to specify:
 - Server configuration, e.g. port number, HTTP or HTTPS
 - Any standard operations for all requests
 - E.g. handle different MIME types, marshall and unmarshall program data structures, check authentication
 - Sometimes called “middleware” (esp. in Javascript servers)
 - URL path and method -> handler mapping
- Each handler will need to:
 - Interpret the request (e.g. body, parameters) and perform the operation
 - Specify the response, potentially including HTTP status, headers and body
 - Frameworks may map Java Exceptions to HTTP errors, and marshall return values

Java Server Configuration Methods

- Depending on the framework used (and how it is used), configuration may be done using:
 - Configuration files, e.g. Servlet web.xml file, Spring configuration files,
 - Application-specific code calling configuration APIs at runtime, or
 - Code annotations, e.g. Spring Boot,

This class handles REST-style requests

```
@RestController  
public class HelloController {
```

This method handles GET on "/"

```
@GetMapping("/")  
public String index() {  
    return "Greetings from Spring Boot!";  
}
```

```
}
```

API-specific servers

- As with clients, you could write this by hand following the API spec, or
- OpenAPI code generators can create a complete skeleton server automatically from the OAD (for certain languages & frameworks)
 - e.g. (from the Mailbox API example)

```
public interface MessageApi {  
...  
    @Operation(summary = "Post a new message", description = "Post a new message", tags={  
"mailbox" })  
    ...  
    @RequestMapping(value = "/message",  
        consumes = { "application/json" },  
        method = RequestMethod.POST)  
    ResponseEntity<Void> messagePost(@Parameter(in = ParameterIn.DEFAULT, description = "Create  
a new message in the mailbox", schema=@Schema()) @Valid @RequestBody Message body  
);
```


Summary

- HTTP APIs are defined in terms of allowed combinations of HTTP method, path, query parameters, request body and response, plus authentication
- OpenAPI (formerly Swagger) is a widely used standard for defining HTTP APIs
 - OpenAPI Descriptions can be used in various tools e.g. to create API-specific client libraries or skeleton server implementations
- Form- & query-style web APIs use HTTP method GET or POST and specify the operation to perform in the path
- REST (REpresentational State Transfer) start by defining URLs for possible resources or entities, and use HTTP methods (GET, POST, PUT, DELETE) to specify the operation to perform on that resource (read, create, write, delete)

Next steps

- Lecture 11: Physical & Architectural models
- Lab 6: Java HTTP examples